API仕様

ベースURL

```
開発環境: http://localhost:3000/api
本番環境: https://your-domain.com/api
```

認証方式

JWT（JSON Web Token）

保護されたエンドポイントでは、リクエストヘッダーにJWTトークンを含める必要があります：

```
Authorization: Bearer <JWT_TOKEN>
```

トークンの取得

`POST /api/auth/login` エンドポイントでトークンを取得できます。

トークンの有効期限

- 有効期限: 1日
- 期限切れ後は再ログインが必要

エンドポイント一覧

認証API（/api/auth）

POST /api/auth/signup

ユーザー登録

認証: 不要

リクエストボディ:

```
{
  "email": "user@example.com",
  "user_name": "山田太郎",
  "password": "securePassword123"
}
```

レスポンス（成功: 201）:

```
{
  "message": "ユーザー登録が完了しました。",
  "userId": 1
}
```

バリデーション:

- メールアドレス: 必須、正しい形式
- ユーザー名: 必須、25文字以内
- パスワード: 8文字以上16文字以内、半角英数字と記号のみ

エラーレスポンス:

- `400`: メールアドレス・パスワード・ユーザー名のバリデーションエラー
- `409`: メールアドレスが既に登録済み
- `500`: サーバーエラー

POST /api/auth/login

ログイン

認証: 不要

リクエストボディ:

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

レスポンス（成功: 200）:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "user_name": "山田太郎",
    "email": "user@example.com",
    "bio": null,
    "avatar_url": "uploads/avatar-1234567890.jpg",
    "default_view_mode": "grid",
    "default_sort_order": "date_logged"
  }
}
```

エラーレスポンス:

- `401`: メールアドレスまたはパスワードが正しくない
- `500`: サーバーエラー

GET /api/auth/me

ログイン中のユーザー情報を取得

認証: 必要

リクエストヘッダー:

```
Authorization: Bearer <JWT_TOKEN>
```

レスポンス（成功: 200）:

```
{
  "user": {
    "id": 1,
    "user_name": "山田太郎",
    "email": "user@example.com",
    "bio": null,
    "avatar_url": "uploads/avatar-1234567890.jpg",
    "default_view_mode": "grid",
    "default_sort_order": "date_logged"
```

```
  }
}
```

エラーレスポンス:

- `401`: 認証トークンが無効
- `404`: ユーザーが見つからない
- `500`: サーバーエラー

記録API（/api/records）

すべてのエンドポイントで認証が必要です。

POST /api/records

記録の作成

認証: 必要

リクエスト:

- Content-Type: `multipart/form-data`
- ボディ:
  - `title` (string, 任意): 記録のタイトル
  - `description` (string, 任意): 記録の説明
  - `date_logged` (string, 必須): 記録日付（YYYY-MM-DD形式）
  - `category_id` (integer, 任意): カテゴリーID
  - `image` (file, 任意): 画像ファイル
  - `show_in_timeline` (boolean/string, 任意): スレッドタイムラインに表示するか（デフォルト: true）

レスポンス（成功: 201）:

```
{
  "message": "記録が作成されました。",
  "recordId": 10,
  "imageUrl": "uploads/1234567890-filename.jpg"
}
```

エラーレスポンス:

- `400`: 日付が未入力、または画像アップロードエラー
- `401`: 認証トークンが無効
- `500`: サーバーエラー

GET /api/records

記録の一覧取得

認証: 必要

クエリパラメータ:

- `category_id` (integer, 任意): カテゴリーIDでフィルター

レスポンス（成功: 200）:

```
[
  {
    "id": 10,
    "title": "今日の振り返り",
    "description": "良い一日だった",
    "date_logged": "2025-12-07",
    "image_url": "uploads/1234567890-filename.jpg",
    "category_id": 1,
    "category_name": "仕事",
    "show_in_timeline": 1,
    "created_at": "2025-12-07T10:30:00.000Z"
  },
  {
    "id": 9,
    "title": "昨日の記録",
    "description": "忙しかった",
    "date_logged": "2025-12-06",
    "image_url": "uploads/1234567891-filename.jpg",
    "category_id": null,
    "category_name": null,
    "show_in_timeline": 1,
    "created_at": "2025-12-06T15:20:00.000Z"
  }
]
```

注意:

- ログイン中のユーザーの記録のみが返されます
- 削除済み（invalidation_flag=1）の記録は含まれません
- 作成日時の降順でソートされます

GET /api/records/:id

特定の記録を取得

認証: 必要

パスパラメータ:

- `id` (integer): 記録ID

レスポンス（成功: 200）:

```
{
  "id": 10,
  "title": "今日の振り返り",
  "description": "良い一日だった",
  "date_logged": "2025-12-07",
  "image_url": "uploads/1234567890-filename.jpg",
  "category_id": 1,
  "category_name": "仕事",
  "show_in_timeline": 1,
  "created_at": "2025-12-07T10:30:00.000Z"
}
```

エラーレスポンス:

- `401`: 認証トークンが無効
- `404`: 記録が見つからない、またはアクセス権限がない
- `500`: サーバーエラー

PUT /api/records/:id

記録の更新

認証: 必要

パスパラメータ:

- `id` (integer): 記録ID

リクエスト:

- Content-Type: `multipart/form-data`
- ボディ:
  - `title` (string, 任意): 記録のタイトル
  - `description` (string, 任意): 記録の説明
  - `date_logged` (string, 必須): 記録日付（YYYY-MM-DD形式）
  - `category_id` (integer, 任意): カテゴリーID（null可）
  - `image` (file, 任意): 新しい画像ファイル（更新する場合）
  - `show_in_timeline` (boolean/string, 任意): スレッドタイムラインに表示するか

レスポンス（成功: 200）:

```
{
  "message": "記録が更新されました。",
  "imageUrl": "uploads/1234567890-filename.jpg"
}
```

エラーレスポンス:

- `400`: 日付が未入力、または画像アップロードエラー
- `401`: 認証トークンが無効
- `404`: 記録が見つからない、または更新権限がない
- `500`: サーバーエラー

DELETE /api/records/:id

記録の削除（論理削除）

認証: 必要

パスパラメータ:

- `id` (integer): 記録ID

レスポンス（成功: 200）:

```
{
  "message": "記録が削除されました。"
}
```

エラーレスポンス:

- `401`: 認証トークンが無効
- `404`: 記録が見つからない、または削除権限がない
- `500`: サーバーエラー

注意: 物理削除ではなく、`invalidation_flag=1` に設定し、`delete_at` に現在時刻を記録する論理削除です。

カテゴリーAPI（/api/categories）

すべてのエンドポイントで認証が必要です。

GET /api/categories

カテゴリーの一覧取得

認証: 必要

レスポンス（成功: 200）:

```
{
  "categories": [
    {
      "id": 1,
      "name": "仕事",
      "created_at": "2025-12-01T10:00:00.000Z",
      "updated_at": "2025-12-01T10:00:00.000Z"
    },
    {
      "id": 2,
      "name": "プライベート",
      "created_at": "2025-12-02T10:00:00.000Z",
      "updated_at": "2025-12-02T10:00:00.000Z"
    }
  ]
}
```

POST /api/categories

カテゴリーの作成

認証: 必要

リクエストボディ:

```
{
  "name": "仕事"
}
```

レスポンス（成功: 201）:

```
{
  "message": "カテゴリーを作成しました。",
  "category": {
    "id": 1,
    "name": "仕事",
    "created_at": "2025-12-07T10:00:00.000Z",
    "updated_at": "2025-12-07T10:00:00.000Z"
  }
}
```

エラーレスポンス:

- `400`: カテゴリー名が未入力
- `401`: 認証トークンが無効
- `500`: サーバーエラー

PUT /api/categories/:id

カテゴリーの更新

認証: 必要

パスパラメータ:

- `id` (integer): カテゴリーID

リクエストボディ:

```
{
  "name": "仕事（更新）"
}
```

レスポンス（成功: 200）:

```
{
  "message": "カテゴリーを更新しました。",
  "category": {
    "id": 1,
    "name": "仕事（更新）",
    "created_at": "2025-12-07T10:00:00.000Z",
    "updated_at": "2025-12-07T11:00:00.000Z"
  }
}
```

エラーレスポンス:

- `400`: カテゴリー名が未入力
- `401`: 認証トークンが無効
- `404`: カテゴリーが見つからない
- `500`: サーバーエラー

DELETE /api/categories/:id

カテゴリーの削除

認証: 必要

パスパラメータ:

- `id` (integer): カテゴリーID

レスポンス（成功: 200）:

```
{
  "message": "カテゴリーを削除しました。"
}
```

エラーレスポンス:

- `401`: 認証トークンが無効
- `404`: カテゴリーが見つからない
- `500`: サーバーエラー

注意: 物理削除です。関連する記録の `category_id` は NULL になります。

ユーザーAPI（/api/users）

すべてのエンドポイントで認証が必要です。

GET /api/users/me

ユーザー情報の取得（アバター・フォロー数含む）

認証: 必要

レスポンス（成功: 200）:

```
{
  "user": {
    "id": 1,
    "user_name": "山田太郎",
    "email": "user@example.com",
    "bio": null,
    "avatar_url": "uploads/avatar-1234567890.jpg",
    "default_view_mode": "grid",
    "default_sort_order": "date_logged",
    "following_count": 5,
    "follower_count": 3
  }
}
```

エラーレスポンス:

- `401`: 認証トークンが無効
- `404`: ユーザーが見つからない
- `500`: サーバーエラー

PUT /api/users/profile

プロフィール情報の更新

認証: 必要

リクエスト:

- Content-Type: `multipart/form-data`
- ボディ:
  - `user_name` (string, 任意): ユーザー名（25文字以内）

- `bio` (string, 任意): 自己紹介
- `visibility` (string, 任意): 公開設定（`public` | `private`）
- `avatar` (file, 任意): アバター画像

レスポンス（成功: 200）:

```
{
  "message": "プロフィールを更新しました",
  "user": {
    "id": 1,
    "user_name": "山田花子",
    "email": "user@example.com",
    "bio": "自己紹介文",
    "avatar_url": "uploads/avatar-1234567890.jpg"
  }
}
```

エラーレスポンス:

- `400`: 画像アップロードエラー、ユーザー名25文字超過
- `401`: 認証トークンが無効
- `500`: サーバーエラー

注意: メールアドレスは変更できません。

PUT /api/users/me/settings

表示設定の更新（一覧のデフォルト表示形式・並び順）

認証: 必要

リクエストボディ:

```
{
  "default_view_mode": "grid",
  "default_sort_order": "date_logged"
}
```

- `default_view_mode`: `grid` | `list` | `booklist` | `tile`
- `default_sort_order`: `date_logged` | `created_at`

レスポンス（成功: 200）: 更新後のユーザー情報（GET /api/users/me と同形式）を返す。

GET /api/users/search

ユーザー検索（公開は部分一致、非公開は検索キー完全一致時のみ）

認証: 必要

クエリパラメータ:

- `q` (string): 検索文字列
- `limit` (integer, 任意): 取得件数（デフォルト50、最大50）

レスポンス（成功: 200）:

```
{
  "users": [
    {
      "id": 2,
      "user_name": "山田花子",
      "bio": null,
      "avatar_url": "uploads/avatar-xxx.jpg",
      "is_following": false
    }
  ]
}
```

GET /api/users/me/following

フォロー中一覧

認証: 必要

レスポンス（成功: 200）:

```
{
  "users": [
    {
      "id": 2,
      "user_name": "山田花子",
      "bio": null,
```

```
      "avatar_url": "uploads/avatar-xxx.jpg"
    }
  ]
}
```

GET /api/users/me/followers

フォロワー一覧（自分がそのユーザーをフォローしているか含む）

認証: 必要

レスポンス（成功: 200）:

```
{
  "users": [
    {
      "id": 3,
      "user_name": "佐藤一郎",
      "bio": null,
      "avatar_url": null,
      "is_following": true
    }
  ]
}
```

GET /api/users/:id

他ユーザーの公開プロフィール取得

認証: 必要

パスパラメータ:

- `id` (integer): ユーザーID

レスポンス（成功: 200）:

```
{
  "user": {
    "id": 2,
    "user_name": "山田花子",
    "bio": "自己紹介",
```

```
    "avatar_url": "uploads/avatar-xxx.jpg",
    "is_following": false
  }
}
```

GET /api/users/:id/records

他ユーザーの投稿一覧取得（プロフィール画面用）

認証: 必要

パスパラメータ:

- `id` (integer): ユーザーID

レスポンス（成功: 200）:

```
{
  "records": [
    {
      "id": 10,
      "title": "今日の振り返り",
      "description": "良い一日だった",
      "date_logged": "2025-12-07",
      "image_url": "uploads/xxx.jpg",
      "category_id": 1,
      "category_name": "仕事",
      "show_in_timeline": 1,
      "created_at": "2025-12-07T10:30:00.000Z"
    }
  ]
}
```

フォローAPI（/api/follows）

すべてのエンドポイントで認証が必要です。

POST /api/follows

フォローする

認証: 必要

リクエストボディ:

```
{
  "following_id": 2
}
```

レスポンス（成功: 201）:

```
{
  "message": "フォローしました。",
  "following": true
}
```

エラーレスポンス:

- `400`: following_id が不正、自分自身はフォロー不可
- `404`: ユーザーが見つからない
- `500`: サーバーエラー

DELETE /api/follows/:following_id

フォロー解除

認証: 必要

パスパラメータ:

- `following_id` (integer): フォロー解除するユーザーID

レスポンス（成功: 200）:

```
{
  "message": "フォローを解除しました。",
  "following": false
}
```

スレッド・タイムラインAPI（/api/threads）

すべてのエンドポイントで認証が必要です。

GET /api/threads/timeline

フォロー中ユーザーの直近7日間の記録を取得

認証: 必要

レスポンス（成功: 200）:

```
{
  "records": [
    {
      "id": 10,
      "title": "今日の振り返り",
      "description": "良い一日だった",
      "date_logged": "2025-12-07",
      "image_url": "uploads/xxx.jpg",
      "category_id": 1,
      "category_name": "仕事",
      "author_id": 2,
      "author_name": "山田花子",
      "author_avatar_url": "uploads/avatar-xxx.jpg",
      "created_at": "2025-12-07T10:30:00.000Z"
    }
  ]
}
```

注意: `show_in_timeline=1` の記録のみ、フォロー中のユーザーの直近7日間分が返されます。

その他のAPI

GET /api/test-db

データベース接続テスト

認証: 不要

レスポンス（成功: 200）:

```
{
  "message": "データベース接続に成功しました！",
  "solution": 2
}
```

画像URLの取得

アップロードされた画像は、以下のURLでアクセスできます：

```
http://<server-host>:3000/<image_url>
```

例:

```
http://localhost:3000/uploads/1234567890-filename.jpg
http://localhost:3000/uploads/categories/category-1234567890.jpg
http://localhost:3000/uploads/avatar-1234567890.jpg
```

エラーレスポンス形式

すべてのエラーは以下の形式で返されます：

```
{
  "message": "エラーメッセージ",
  "error": "詳細なエラー情報（開発環境のみ）"
}
```

HTTPステータスコード

- `200`：成功
- `201`：作成成功
- `400`：リクエストエラー（バリデーションエラーなど）
- `401`：認証エラー（トークンが無効など）
- `404`：リソースが見つからない
- `409`：競合エラー（メールアドレス重複など）
- `500`：サーバーエラー

レート制限

現状、レート制限は実装されていません。将来の拡張として検討予定です。

CORS設定

開発環境では、すべてのオリジンからのアクセスを許可しています。 本番環境では、適切なオリジンを設定してください。